Sauce Demo QA Strategy – Ali Bdaer

Requirements and documentation

I would first confirm the expected sorting and purchase behaviour and how to identify each staging update. I would keep one QA note and checklist in the feature ticket, with links to bugs and test results.

1 Questions before testing

Questions for Product

- **What should the price option change?** Does “filter by price” mean sorting the displayed products, with no products removed and no prices changed?
- **Are equal prices allowed?** The brief says “strictly ascending”. I would confirm whether two products with the same price are acceptable before deciding what should pass or fail.
- **What counts as a successful purchase?** I would confirm the expected cart contents, prices and totals, and what must happen for the order to be considered complete.
- **What is included in this change?** Which checkout field validations, browsers and account types must be covered?

Questions for Development and the CI owner

- **How will I know what changed and which version is on staging?** A short deployment message with the version number and links to the changed tickets or commits would give me a starting point.
- **Can a deployment happen while tests are running?** How can we detect a version change during a run? I would treat that run as inconclusive and agree when to rerun affected checks within the CI budget.
- **What test data can I use?** Are the account and cart shared with other people or tests? How do I reset the cart, and can product data change during a run?
- **Which page elements can tests rely on?** Without dedicated test IDs, I would ask which existing IDs, names or labels are stable, and what shows that sorting has finished.
- **What is the exact CI time limit?** Does it include setup, browser startup and report upload? Which existing checks can I reuse?
- **Who reviews a failed check, and what happens next?** Since the update is already on staging, who investigates and decides whether to fix the issue, roll back the update or block its release to production?

I would record unanswered questions in the feature ticket with a named owner. I would agree the sorting and purchase rules with Product before writing assertions for them.

Working with frequent deployments

For each deployment, I would prioritise login, price sorting, the correct cart contents and one completed purchase. I would reuse existing checks and measure the full job against the CI limit. Additional browsers and lower-priority checkout cases would run in a separate scheduled job, if the CI owner agrees. I would update affected checklist rows and link each result to the version tested.

2 Documentation I would keep

I would keep three linked records in the team's existing tools.

A short QA note with a test checklist

Where: In the feature ticket. QA maintains it and Product confirms the expected behaviour.

Note fields: Scope, agreed rules, open questions and owners.
Checklist fields: Rule, risk, check, expected result, priority, status, and links to results or bugs.

The checklist links each agreed rule to its check and result. That provides traceability in the same place. For example, these are proposed checks, not execution results:

Risk	Check and expected result	Priority
Incorrect price order	Select low-to-high. All displayed prices follow the agreed rule, and the same products and prices remain.	High
Wrong item after sorting	Add a product after sorting. Its name, price and quantity match in the cart and order summary.	High
Purchase cannot finish	Complete checkout with valid data. The order has the correct items and total, and a confirmation is shown.	Highest

A bug report

Where: The existing bug tracker, linked to the feature and the relevant test run.

Fields: Clear title; environment, build/version and browser; test data; steps; expected result; actual result; severity and business impact; agreed priority; screenshots/videos.

If the expected behaviour is still unclear, I would record a question first instead of opening a bug based on an assumption.

A short run summary

Where: CI stores automated results. The feature ticket links to the run and records manual results, the next action and its owner.

Fields: Date, environment and build; checks run and not run; passed, failed or blocked results; duration; evidence and bug links; next action and owner.

What I would skip

For this feature, I would skip a long standalone test plan, duplicated test-case spreadsheets and a separate traceability matrix. The QA note and checklist already contain the planning and links. I would use the generated CI results instead of copying passing results into another report.

3 Automation Project

The Selenium and TestNG test uses Page Object Model and covers login and price sorting. It follows the brief's strict rule: each price must be greater than the previous one, so equal prices fail.

Repository: https://github.com/alibdaer/Automation-Task-Ali-Bdaer

Requirements: IntelliJ IDEA, JDK 17, and Google Chrome.

1. Clone the repository, or use Code > Download ZIP and extract it.
2. In IntelliJ IDEA, select File > Open, choose pom.xml, then select Open as Project.
3. Set the Project SDK to JDK 17 and allow Maven dependencies to finish loading.
4. Open src/test/java/PriceSortTest.java and click the green Run icon beside the class or run testng.xml file.